

Discrete Project

Intended for DR. Sherine Magdy

Student Name

Abdullah Ali Elwan Ahmed

Student Code: 12500626



The Logical Equivalence Proof

Our project focuses on proving the logical equivalence of a complex Boolean expression:

$$((P \wedge \neg Q) \vee (R \oplus C)) \rightarrow (P \leftrightarrow C) = ((\neg P \vee Q) \vee (P \leftrightarrow C)) \wedge ((R \leftrightarrow C) \vee (P \leftrightarrow C))$$

This involves a detailed truth table analysis for both the Left-Hand Side (L.H.S) and Right-Hand Side (R.H.S) of the equation.



L.H.S: Truth Table Analysis

The Left-Hand Side of the equation is: $((P \wedge \neg Q) \vee (R \oplus C)) \rightarrow (P \leftrightarrow C)$

P	Q	R	C	$\neg Q$	$(P \wedge \neg Q)$	$(R \oplus C)$	$\frac{(P \wedge \neg Q)}{(R \oplus C)}$	$(P \leftrightarrow C)$	Result
T	T	T	T	F	F	F	F	T	T
T	T	T	F	F	F	T	T	F	F
T	T	F	T	F	F	T	T	T	T
T	T	F	F	F	F	F	F	F	T
T	F	T	T	T	T	F	T	T	T
T	F	T	F	T	T	T	T	F	F
T	F	F	T	T	T	T	T	T	T
T	F	F	F	T	T	F	T	F	F
F	T	T	T	F	F	F	F	F	T
F	T	T	F	F	F	T	T	T	T
F	T	F	T	F	F	T	T	F	F
F	T	F	F	F	F	F	F	T	T
F	F	T	T	T	F	F	F	F	T
F	F	T	F	T	F	T	T	T	T
F	F	F	T	T	F	T	T	F	F
F	F	F	F	T	F	F	F	T	T

R.H.S: Truth Table Analysis

The Right-Hand Side of the equation is: $((\neg P \vee Q) \vee (P \leftrightarrow C)) \wedge ((R \leftrightarrow C) \vee (P \leftrightarrow C))$

P	Q	R	C	$\neg P$	$(\neg P \vee Q)$	$(P \leftrightarrow C)$	$(\neg P \vee Q) \vee (P \leftrightarrow C)$	$(R \leftrightarrow C)$	$(R \leftrightarrow C) \vee (P \leftrightarrow C)$	Result
T	T	T	T	F	T	T	T	T	T	T
T	T	T	F	F	T	F	T	F	F	F
T	T	F	T	F	T	T	T	F	T	T
T	T	F	F	F	T	F	T	T	T	T
T	F	T	T	F	F	T	T	T	T	T
T	F	T	F	F	F	F	F	F	F	F
T	F	F	T	F	F	T	T	F	T	T
T	F	F	F	F	F	F	F	T	T	F
F	T	T	T	T	T	F	T	T	T	T
F	T	T	F	T	T	T	T	F	T	T
F	T	F	T	T	T	F	T	F	F	F
F	T	F	F	T	T	T	T	T	T	T
F	F	T	T	T	T	F	T	T	T	T
F	F	T	F	T	T	T	T	F	T	T
F	F	F	T	T	T	F	T	F	F	F
F	F	F	F	T	T	T	T	T	T	T

L.H.S: Numeric Truth Table

Representing truth values as 1 (True) and 0 (False) for the L.H.S expression.

P	Q	R	C	$\neg Q$	$(P \wedge \neg Q)$	$(R \oplus C)$	$(P \wedge \neg Q) \vee (R \oplus C)$	$(P \leftrightarrow C)$	Result
1	1	1	1	0	0	0	0	1	1
1	1	1	0	0	0	1	1	0	0
1	1	0	1	0	0	1	1	1	1
1	1	0	0	0	0	0	0	0	1
1	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	0	0
1	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	0	0
0	1	1	1	0	0	0	0	0	1
0	1	1	0	0	0	1	1	1	1
0	1	0	1	0	0	1	1	0	0
0	1	0	0	0	0	0	0	1	1
0	0	1	1	1	0	0	0	0	1
0	0	1	0	1	0	1	1	1	1
0	0	0	1	1	0	1	1	0	0
0	0	0	0	1	0	0	0	1	1

R.H.S: Numeric Truth Table

Representing truth values as 1 (True) and 0 (False) for the R.H.S expression.

P	Q	R	C	$\neg P$	$(\neg P \vee Q)$	$(P \leftrightarrow C)$	$(\neg P \vee Q) \vee (P \leftrightarrow C)$	$(R \leftrightarrow C)$	$(R \leftrightarrow C) \vee (P \leftrightarrow C)$	Result
1	1	1	1	0	1	1	1	1	1	1
1	1	1	0	0	1	0	1	0	0	0
1	1	0	1	0	1	1	1	0	1	1
1	1	0	0	0	1	0	1	1	1	1
1	0	1	1	0	0	1	1	1	1	1
1	0	1	0	0	0	0	0	0	0	0
1	0	0	1	0	0	1	1	0	1	1
1	0	0	0	0	0	0	0	1	1	0
0	1	1	1	1	1	0	1	1	1	1
0	1	1	0	1	1	1	1	0	1	1
0	1	0	1	1	1	0	1	0	0	0
0	1	0	0	1	1	1	1	1	1	1
0	0	1	1	1	1	0	1	1	1	1
0	0	1	0	1	1	1	1	0	1	1
0	0	0	1	1	1	0	1	0	0	0
0	0	0	0	1	1	1	1	1	1	1

C++ Implementation for Truth Table Generation

A C++ program was developed to automate the generation of truth tables, ensuring accuracy and efficiency in verifying the logical equivalence.

```
#include <iostream>
#include <string>
using namespace std;

char transformer(int value){
    if(value==1) return 'T';
    else return 'F';
}

int main() {
    int p[16]={1,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0};
    int q[16]={1,1,1,1,0,0,0,0,1,1,1,1,0,0,0,0};
    int r[16]={1,1,0,0,1,1,0,0,1,1,0,0,1,1,0,0};
    int c[16]={1,0,1,0,1,0,1,0,1,0,1,0,1,0,1,0};

    int nq[16];
    for(int i=0; i<16; i++){ nq[i]=!q[i]; }

    int pAndNq[16];
    for(int i=0; i<16; i++){ pAndNq[i]=p[i]&& nq[i]; }

    int rExC[16];
    for(int i=0; i<16; i++){ rExC[i]=r[i]^c[i]; }

    int pAndNqOrRExC[16];
    for(int i=0; i<16; i++){ pAndNqOrRExC[i]=pAndNq[i] | rExC[i]; }

    int pBicondC[16];
    for(int i=0; i<16; i++){ pBicondC[i]=(p[i] == c[i]); }

    int result[16];
    for(int i=0; i<16; i++){ result[i]=(!pAndNqOrRExC[i]) | pBicondC[i]; }

    int nP[16];
    for(int i=0; i<16; i++){ nP[i]=!p[i]; }

    int nPOrQ[16];
    for(int i=0; i<16; i++){ nPOrQ[i]=nP[i] | q[i]; }

    int pIFFc[16];
    for(int i=0; i<16; i++){ pIFFc[i]= (p[i] == c[i]); }

    int nPOrQOrPIFFc[16];
    for(int i=0; i<16; i++){ nPOrQOrPIFFc[i]=nPOrQ[i] | pIFFc[i]; }

    int rIFFc[16];
    for(int i=0; i<16; i++){ rIFFc[i]= (r[i] == c[i]); }

    int rIFFcOrPIFFc[16];
    for(int i=0; i<16; i++){ rIFFcOrPIFFc[i]=rIFFc[i] | pIFFc[i]; }

    int nPOrQOrPIFFcANdRIFFcOrPIFFc[16];
    for(int i=0; i<16; i++){ nPOrQOrPIFFcANdRIFFcOrPIFFc[i]=nPOrQOrPIFFc[i]&& rIFFcOrPIFFc[i]; }
```

C++ Implementation for Truth Table Generation

A C++ program was developed to automate the generation of truth tables, ensuring accuracy and efficiency in verifying the logical equivalence.

```
// ----- PRINT TRUTH TABLE (FIRST) -----

cout << "+----+----+----+----+----+-----+-----+-----+-----+-----+-----+\\n";
cout << "| P | Q | R | C | ¬Q | P∧¬Q | R⊕C | (P∧¬Q) ∨ (R⊕C) | PC | RESULT |\\n";
cout << "+----+----+----+----+----+-----+-----+-----+-----+-----+-----+\\n";

for(int i=0; i<16; i++){
    cout << "| " << transformer(p[i]) << " | "
        << transformer(q[i]) << " | "
        << transformer(r[i]) << " | "
        << transformer(c[i]) << " | "
        << transformer(nq[i]) << " | "
        << transformer(pAndNq[i]) << " | "
        << transformer(rExC[i]) << " | "
        << transformer(pAndNqOrRExC[i]) << " | "
        << transformer(pBicondC[i]) << " | "
        << transformer(result[i])
        << " |\\n";
}

cout << "+----+----+----+----+----+-----+-----+-----+-----+-----+-----+\\n";

// ----- PRINT TRUTH TABLE (SECOND, FIXED) -----
// ((¬P∨Q)∨(PC))∧((RC)∨(PC))cout << "+----+----+----+----+----+-----+-----+-----+-----+-----+-----+\\n";
cout << "| P | Q | R | C | ¬P | ¬P∨Q | PC | (¬P∨Q)∨(PC) | RC | (RC)∨(PC) | RESULT |\\n"; cout << "+----+----+----+----+----+-----+-----+-----+-----+-----+-----+\\n";

for(int i=0; i<16; i++) {
    cout << "| " << transformer(p[i]) << " | "
        << transformer(q[i]) << " | "
        << transformer(r[i]) << " | "
        << transformer(c[i]) << " | "
        << transformer(nP[i]) << " | "
        << transformer(nPOrQ[i]) << " | "
        << transformer(pIFFc[i]) << " | "
        << transformer(nPOrQOrPIFFc[i]) << " | "
        << transformer(rIFFc[i]) << " | "
        << transformer(rIFFcOrPIFFc[i]) << " | "
        << transformer(nPOrQOrPIFFcANdRIFFcOrPIFFc[i])
        << " |\\n";
}

cout << "+----+----+----+----+----+-----+-----+-----+-----+-----+-----+\\n";
return 0; }
```


C++ Implementation for Truth Table Generation

A C++ program was developed to automate the generation of truth tables, ensuring accuracy and efficiency in verifying the logical equivalence [THE OUTPUT].

P	Q	R	C	$\neg P$	$\neg P \vee Q$	$P \oplus C$	$(\neg P \vee Q) \vee (P \oplus C)$	$R \oplus C$	$(R \oplus C) \vee (P \oplus C)$	RESULT
T	T	T	T	F	T	T	T	T	T	T
T	T	T	F	F	T	F	T	F	F	F
T	T	F	T	F	T	T	T	F	T	T
T	T	F	F	F	T	F	T	T	T	T
T	F	T	T	F	F	T	T	T	T	T
T	F	T	F	F	F	F	T	F	F	F
T	F	F	T	F	F	T	T	F	T	T
T	F	F	F	F	F	F	T	T	T	F
F	T	T	T	T	T	F	T	T	T	T
F	T	T	F	T	T	T	T	F	T	T
F	T	F	T	T	T	F	T	F	F	F
F	T	F	F	T	T	T	T	T	T	T
F	F	T	T	T	T	F	T	T	T	T
F	F	T	F	T	T	T	T	F	T	T
F	F	F	T	T	T	F	T	F	F	F
F	F	F	F	T	T	T	T	T	T	T

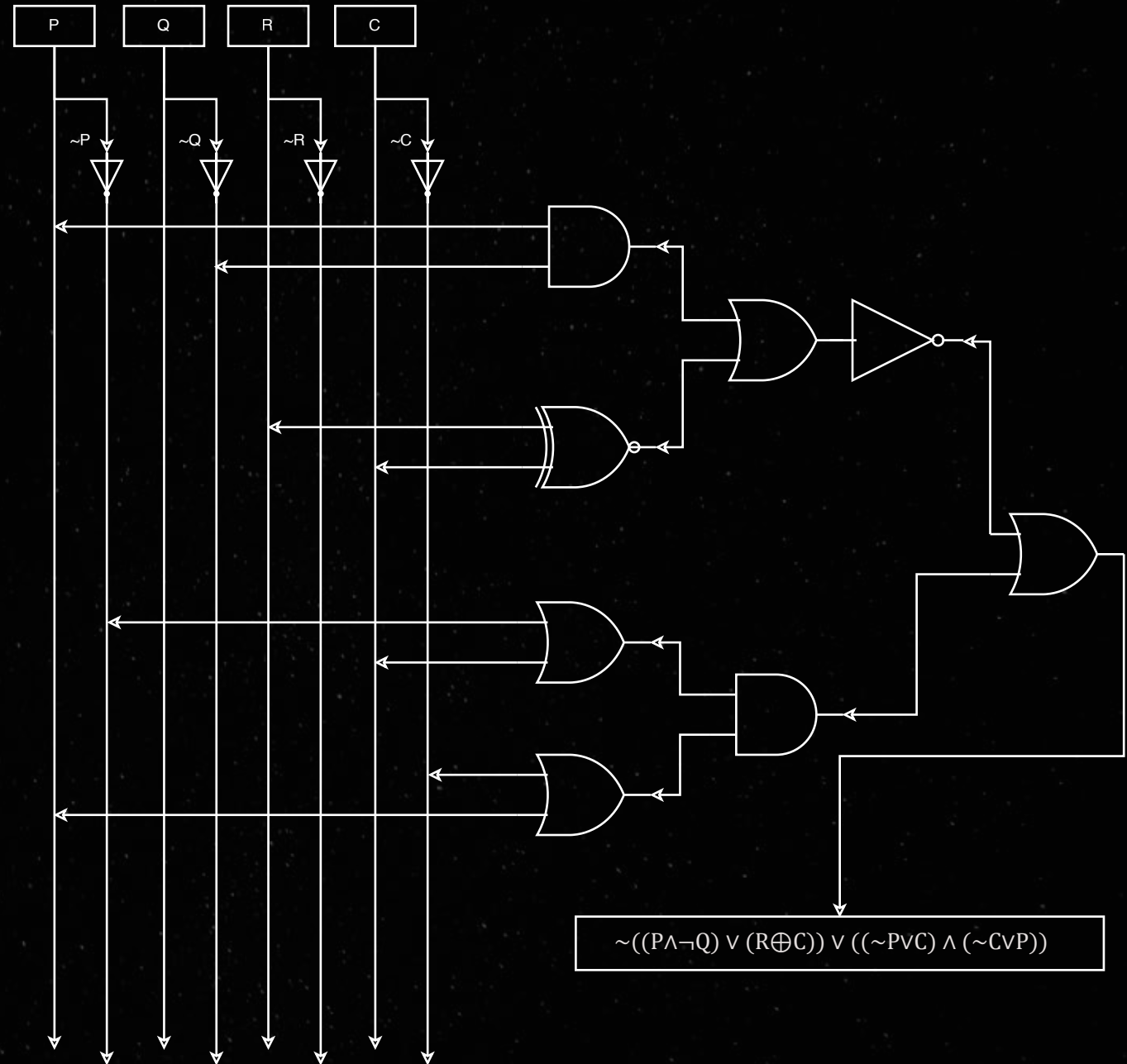
P	Q	R	C	$\neg Q$	$P \wedge \neg Q$	$R \oplus C$	$(P \wedge \neg Q) \vee (R \oplus C)$	$P \oplus C$	RESULT
T	T	T	T	F	F	F	F	T	T
T	T	T	F	F	F	T	T	F	F
T	T	F	T	F	F	T	T	T	T
T	T	F	F	F	F	F	F	F	T
T	F	T	T	T	T	F	T	T	T
T	F	T	F	T	T	T	T	F	F
T	F	F	T	T	T	T	T	T	T
T	F	F	F	T	T	F	T	F	F
F	T	T	T	F	F	F	F	F	T
F	T	T	F	F	F	T	T	T	T
F	T	F	T	F	F	T	T	F	F
F	T	F	F	F	F	F	F	T	T
F	F	T	T	T	F	F	F	F	T
F	F	T	F	T	F	T	T	T	T
F	F	F	T	T	F	T	T	F	F
F	F	F	F	T	F	F	F	T	T

L.H.S Equation Draw

$$\text{L.H.S} = ((P \wedge \neg Q) \vee (R \oplus C)) \rightarrow (P \leftrightarrow C) = \sim((P \wedge \neg Q) \vee (R \oplus C)) \vee (P \leftrightarrow C)$$

$$(P \leftrightarrow C) = (P \rightarrow C) \wedge (C \rightarrow P) = (\sim P \vee C) \wedge (\sim C \vee P)$$

$$\text{Final Result} = \sim((P \wedge \neg Q) \vee (R \oplus C)) \vee ((\sim P \vee C) \wedge (\sim C \vee P))$$



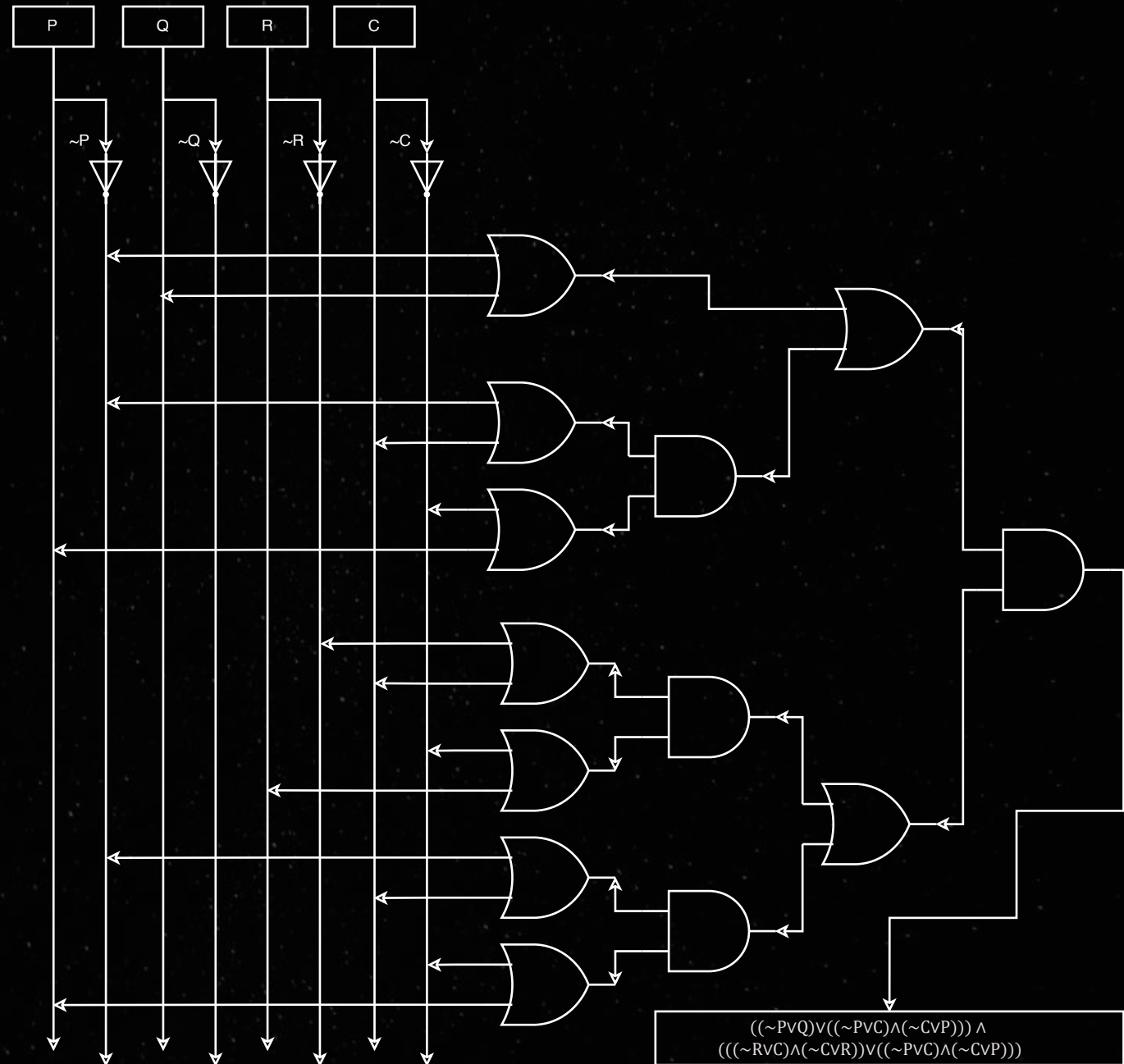
R.H.S Equation Draw

$$\text{R.H.S} = ((\neg P \vee Q) \vee (P \leftrightarrow C)) \wedge ((R \leftrightarrow C) \vee (P \leftrightarrow C))$$

$$(P \leftrightarrow C) = (P \rightarrow C) \wedge (C \rightarrow P) = (\neg P \vee C) \wedge (\neg C \vee P)$$

$$(R \leftrightarrow C) = (R \rightarrow C) \wedge (C \rightarrow R) = (\neg R \vee C) \wedge (\neg C \vee R)$$

$$\text{Final Result} = ((\neg P \vee Q) \vee ((\neg P \vee C) \wedge (\neg C \vee P))) \wedge (((\neg R \vee C) \wedge (\neg C \vee R)) \vee ((\neg P \vee C) \wedge (\neg C \vee P)))$$



Verification of Equivalence



Consistent Results

The final result columns for both L.H.S and R.H.S truth tables are identical.



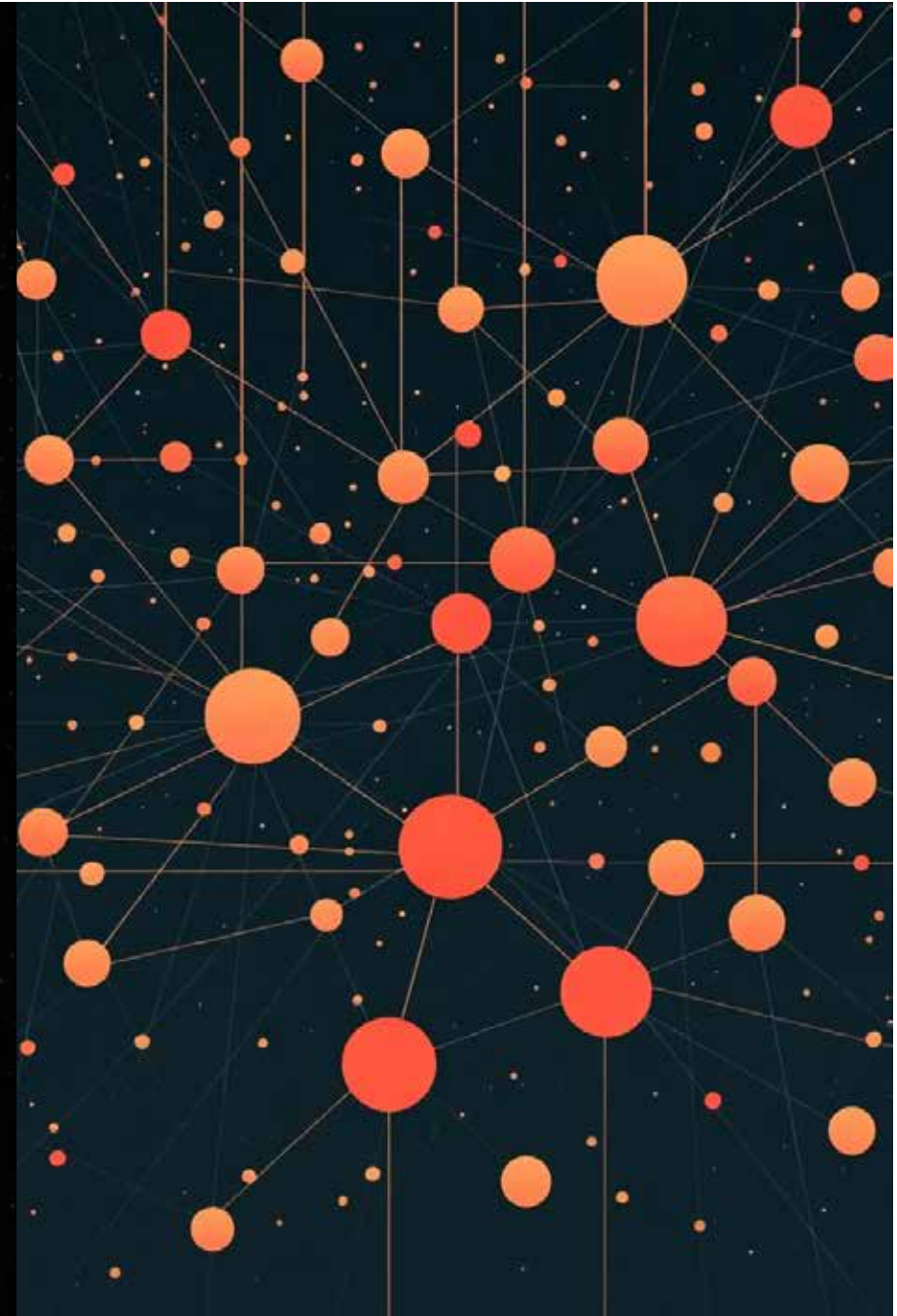
Logical Proof

This consistency mathematically proves the logical equivalence of the given Boolean expressions.



C++ Validation

The C++ program further validates these results, confirming the equivalence through computational means.





Conclusion

Through rigorous truth table analysis and computational verification, we have successfully demonstrated the logical equivalence of the complex Boolean expression. This project highlights the importance of discrete mathematics in computer science and logical reasoning.